

# Practical No. 0

**Aim:** Introduction to DBMS Lab.

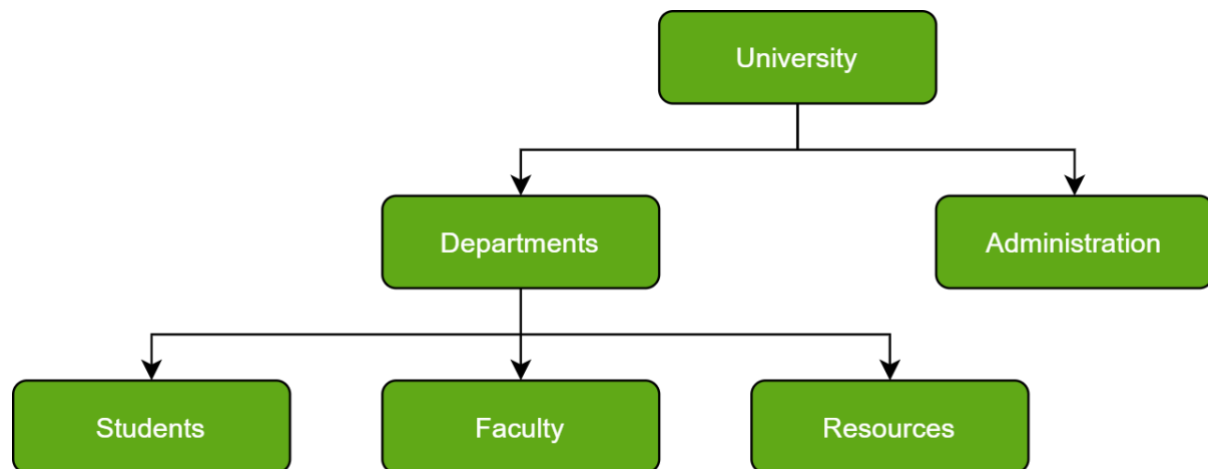
## Theory:

### What is a DBMS?

A Database Management System (DBMS) is software designed to efficiently store, organize, manage, and retrieve data in a structured way. It allows users to create, modify, and query databases while ensuring data integrity, security, and efficient access. Unlike traditional file systems, a DBMS reduces data redundancy, prevents inconsistencies, and simplifies data handling with features like concurrent access, backup, and recovery mechanisms.

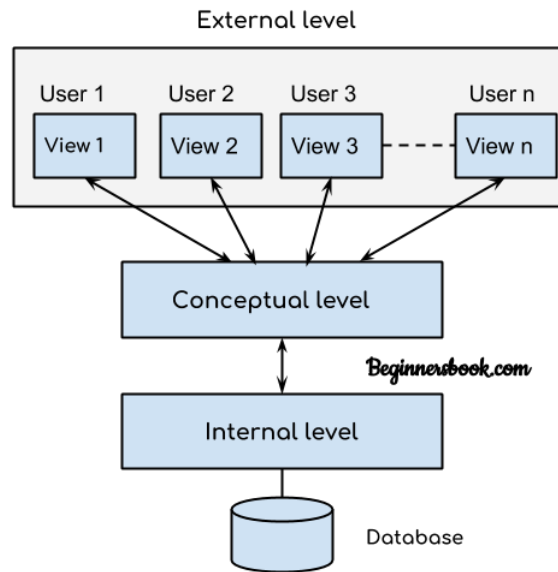
### Example:

A university database can store and manage student information, faculty records, and administrative data, allowing seamless retrieval, insertion, and deletion of information as required.



### Key Features of DBMS

1. **Data Modeling:** Tools to create and modify data models, defining the structure and relationships within the database.
2. **Data Storage and Retrieval:** Efficient mechanisms for storing data and executing queries to retrieve it quickly.
3. **Concurrency Control:** Ensures multiple users can access the database simultaneously without conflicts.



## Types of DBMS

### 1. Relational Database Management System (RDBMS)

[RDBMS](#) organizes data into tables (relations) composed of rows and columns. It uses primary keys to uniquely identify rows and foreign keys to establish relationships between tables. Queries are written in **SQL (Structured Query Language)**, which allows for efficient data manipulation and retrieval.

**Examples:** MySQL, Oracle, Microsoft SQL Server and Postgre SQL.

### 2. NoSQL DBMS

[NoSQL systems](#) are designed to handle **large-scale data** and provide high performance for scenarios where **relational models** might be restrictive. They store data in various non-relational formats, such as **key-value pairs**, **documents**, **graphs**, or **columns**. These flexible data models enable rapid scaling and are well-suited for unstructured or semi-structured data.

**Examples:** MongoDB, Cassandra, DynamoDB and Redis.

### 3. Object-Oriented DBMS (OODBMS)

[OODBMS](#) integrates object-oriented programming concepts into the **database environment**, allowing data to be stored as objects. This approach supports complex data types and relationships, making it ideal for applications requiring advanced data modeling and **real-world simulations**.

**Examples:** ObjectDB, db4o.

### 4. Network DBMS (NDBMS)

Network DBMS uses a flexible model that allows multiple relationships between data entities using **graph-like structures**. Records are connected through **pointers**, forming a **many-to-many** relationship. It enables more complex representations than the hierarchical model and is efficient for large databases with many interconnected relationships.

This model is ideal for applications such as **telecommunications, airline reservation systems**. **Examples:** Integrated Data Store (IDS), IDMS.

## 5. Hierarchical DBMS (HDBMS)

Hierarchical DBMS organizes data in a **tree-like structure**, where data is represented in **parent-child relationships**. Each child record has only **one parent**, forming a strict hierarchy. Navigation follows a **top-down** or **bottom-up** approach.

This model is highly efficient for scenarios where data relationships are well-defined and do not change frequently.

**Examples:** IBM's IMS (Information Management System), Windows Registry.

## Database Languages

Database languages are specialized sets of **commands** and **instructions** used to define, manipulate, and control data within a database. The primary database languages include:

### 1. Data Definition Language (DDL)

[DDL](#) is the short name for Data Definition Language, which deals with database schemas and descriptions, of how the data should reside in the database.

- **CREATE:** to create a database and its objects like (table, index, views, store procedure, function, and triggers)
- **ALTER:** alters the structure of the existing database
- **DROP:** delete objects from the database
- **TRUNCATE:** remove all records from a table, including all spaces allocated for the records are removed
- **RENAME:** rename an object

### 2. Data Manipulation Language (DML)

[DML](#) focuses on manipulating the data stored in the database, enabling users to retrieve, add, update, and delete data.

- **SELECT:** retrieve data from a database

- **INSERT:** insert data into a table
- **UPDATE:** updates existing data within a table
- **DELETE:** Delete all records from a database table
- **MERGE:** UPSERT operation (insert or update)

### 3. Data Control Language (DCL)

[DCL](#) commands manage access permissions, ensuring data security by controlling who can perform certain actions on the database.

- **GRANT:** Provides specific privileges to a user (e.g., SELECT, INSERT).
- **REVOKE:** Removes previously granted permissions from a user.

### 4. Transaction Control Language (TCL)

[TCL](#) commands oversee transactional data to maintain consistency, reliability, and atomicity.

- **ROLLBACK:** Undoes changes made during a transaction.
- **COMMIT:** Saves all changes made during a transaction.
- **SAVEPOINT:** Sets a point within a transaction to which one can later roll back.

### 5. Data Query Language (DQL)

[DQL](#) is a subset of DML, specifically focused on data retrieval.

- **SELECT:** The primary DQL command, used to query data from the database without altering its structure or contents.

### Advantages of DBMS

1. **Data organization:** A DBMS allows for the organization and storage of data in a structured manner, making it easy to retrieve and query the data as needed.
2. **Data integrity:** A DBMS provides mechanisms for enforcing data integrity constraints, such as constraints on the values of data and access controls that restrict who can access the data.
3. **Concurrent access:** A DBMS provides mechanisms for controlling concurrent access to the [database](#), to ensure that multiple users can access the data without conflicting with each other.
4. **Data security:** A DBMS provides tools for managing the security of the data, such as controlling access to the data and encrypting sensitive data.

5. **Backup and recovery:** A DBMS provides mechanisms for backing up and recovering the data in the event of a system failure.

## Disadvantages of DBMS

1. **Complexity:** DBMS can be complex to set up and maintain, requiring specialized knowledge and skills.
2. **Performance overhead:** The use of a DBMS can add overhead to the performance of an application, especially in cases where high levels of concurrency are required.
3. **Scalability:** The use of a DBMS can limit the scalability of an application, since it requires the use of locking and other synchronization mechanisms to ensure data consistency.
4. **Cost:** The cost of purchasing, maintaining and upgrading a DBMS can be high, especially for large or complex systems.
5. **Limited Use Cases:** Not all use cases are suitable for a DBMS, some solutions don't need high reliability, consistency or security and may be better served by other types of data storage.

## Applications of DBMS

1. **Enterprise Information:** Sales, accounting, human resources, Manufacturing, online retailers.
2. **Banking and Finance Sector:** Banks maintaining the customer details, accounts, loans, banking transactions, credit card transactions. Finance: Storing the information about sales and holdings, purchasing of financial stocks and bonds.
3. **University:** Maintaining the information about student course enrolled information, student grades, staff roles.

## Conclusion

Hence, I have successfully.....